

Minicurso:

Programação Orientada a Objetos

- modelando a vida,
o universo e tudo mais -

Prof. Dr. Renê Rodrigues Veloso

E a profecia se cumpriu...



“(...)a programação orientada a objetos será nos anos 80 o que a estruturada era nos anos 70. Todo mundo será a favor dela. Todo fabricante irá promover seus produtos como suportando-a. Todo gerente irá pagar para tê-la. Todo programador irá praticá-la. E ninguém saberá o que ela realmente é (...)”

Nguyen, Van e Hailpern, Brent. “A Generalized Object Model”
ACM SIGPLAN NOTICES - v17, n9 Ed: G. Richard Wexelblat -
September 1982



- ♦ O que é um paradigma?
 - Paradigmas de Programação.
- ♦ Paradima OO
 - Por que POO?
- ♦ Conceitos de POO
 - Objeto, Classe, Instância, Encapsulamento, Modificadores de acesso, Construtores, Mensagens, Relacionamento entre objetos, Herança, Sobreposição e sobrecarga, e Polimorfismo.
- ♦ Conclusão
- ♦ Referências



- O que é um paradigma?
 - Modelo, protótipo. (Dic. Michaelis)
 - “Um conjunto de regras que estabelecem fronteiras e descreve como resolver os problemas dentro destas fronteiras.” (web)
 - É a “filosofia” adotada na construção de software.
- Os paradigmas influenciam nossa percepção; ajudam-nos a organizar e a coordenar a maneira como olhamos para o mundo.



Paradigmas de Programação

- ♦ Atualmente existem 4 paradigmas fundamentais:
 - ♦ Lógico
 - ♦ Funcional
 - ♦ Procedimental
 - ♦ Orientado a Objetos

Paradigma Procedimental

- Visão de modelo baseada em:
entrada – processamento – saída.
- Dados são considerados separadamente das funções (procedimentos).
- As funções (procedimentos) são a parte ativa do programa.
- Dados são repositórios passivos de informação afetados por funções.



- ♦ O termo OO significa **abstrair** o mundo real como uma **coleção de objetos** que incorporam **estrutura de dados** e um **conjunto de operações** que manipulam estes dados.
- ♦ **Programação procedimental:**
Algoritmos + Estruturas de dados = programas.
- ♦ **Programação OO:**
Estruturas de dados + Algoritmos = programas.

Orientação a Objetos



- Na POO um programa é feito de objetos com certas propriedades e operações que eles podem realizar.
- O programador só se importa com o que o objeto expõe;
- Não importa como foi implementado (o que tem dentro), e sim que ele faça o que tem que fazer.



- ♦ Por que POO?
 - ♦ Gerenciar a complexidade crescente dos sistemas sendo construídos nas empresas, viabilizando o trabalho conjunto de grandes equipes o cumprimento de metas de qualidade, prazo e orçamento.
 - ♦ Aumentar a produtividade dos analistas e programadores pela reutilização de código pronto e depurado escrito em sistemas anteriores ou adquiridos no mercado.



Orientação a Objetos



- ♦ Reforçando...
 - ♦ A chave da produtividade da POO é que cada objeto é responsável pela realização de um conjunto de tarefas.
 - ♦ Ao invés de passar dados a procedimentos, requisita-se que os objetos realizem as operações neles próprios.



- ♦ Objeto
- ♦ Encapsulamento
- ♦ Classe
- ♦ Instância
- ♦ Herança
- ♦ Polimorfismo
- ♦ Sobreposição e Sobrecarga
- ♦ Construtores



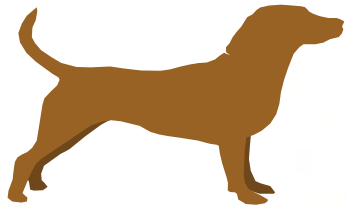
- ♦ Um objeto é a abstração de qualquer 'coisa' do mundo que pode ser manipulado por um programa.
 - ♦ Uma conta-corrente
 - ♦ Um cliente
 - ♦ Uma janela
 - ♦ Um botão
 - ♦ O programa

Objeto



“Qualquer estrutura de dados na qual podemos armazenar dados e métodos que os manipulam” (martin, 1995)

Tudo em OO é um objeto!



Objetos do tipo cachorro



Objetos do tipo casa

Objeto



- Assim como as coisas do mundo real, os objetos possuem **estado**, **comportamento** e **identidade**.
 - Estado: são informações sobre o objeto.
 - Ex.: sua cor, seu peso, o saldo da conta-corrente.
 - Comportamento: são coisas que podem ser feitas com ou pelo objeto.
 - Ex.: fazer um depósito em uma conta, mudar a cor de uma janela.
 - Identidade: é uma identificação que o diferencia de objetos semelhantes (o nome).
 - Ex.: o usuário Rene e o usuário Ramon.

Objeto



- ♦ Exemplo de Objeto: Carro.
 - ♦ Estado:
 - ♦ Fabricante, Nro de portas, Ano, Chassis, Placa, Cor, Combustível, etc.
 - ♦ Comportamento:
 - ♦ Ligar, Desligar, Acelerar, Brecar, Travar portas, Businar, Acender faróis, etc.
 - ♦ Identidade: Meu carro.



Objeto

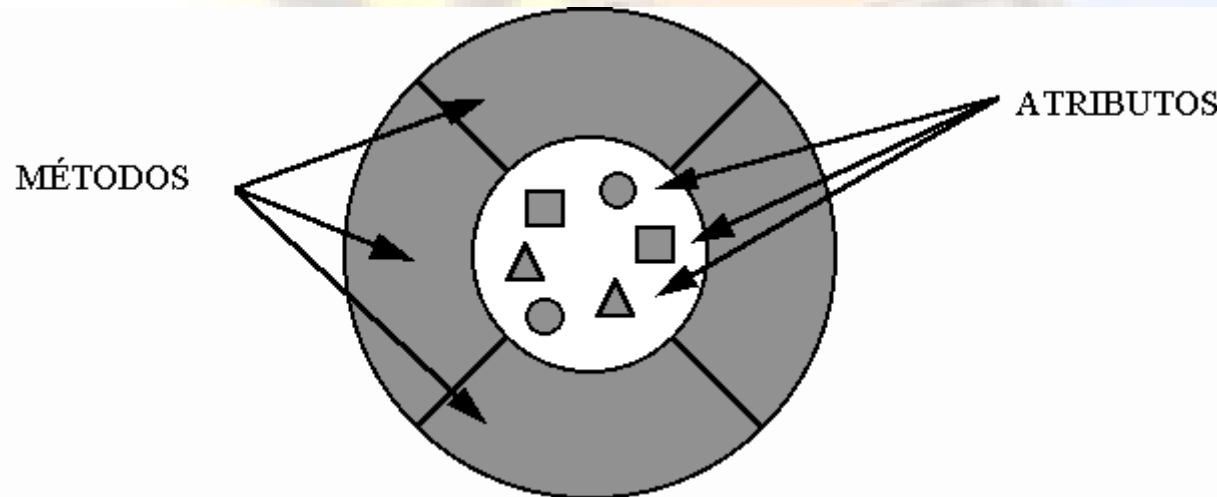


- Os valores que definem o **estado** de um objeto são comumente chamados de **atributos**.
- As funções que definem o **comportamento** de um objeto são frequentemente chamados de **métodos**.

Objeto



- No paradigma procedimental o estado dos programas são as variáveis e o comportamento são as funções/procedimentos.
- Na POO, os atributos e métodos são reunidos em objetos:





Objeto = atributos + métodos

Classe

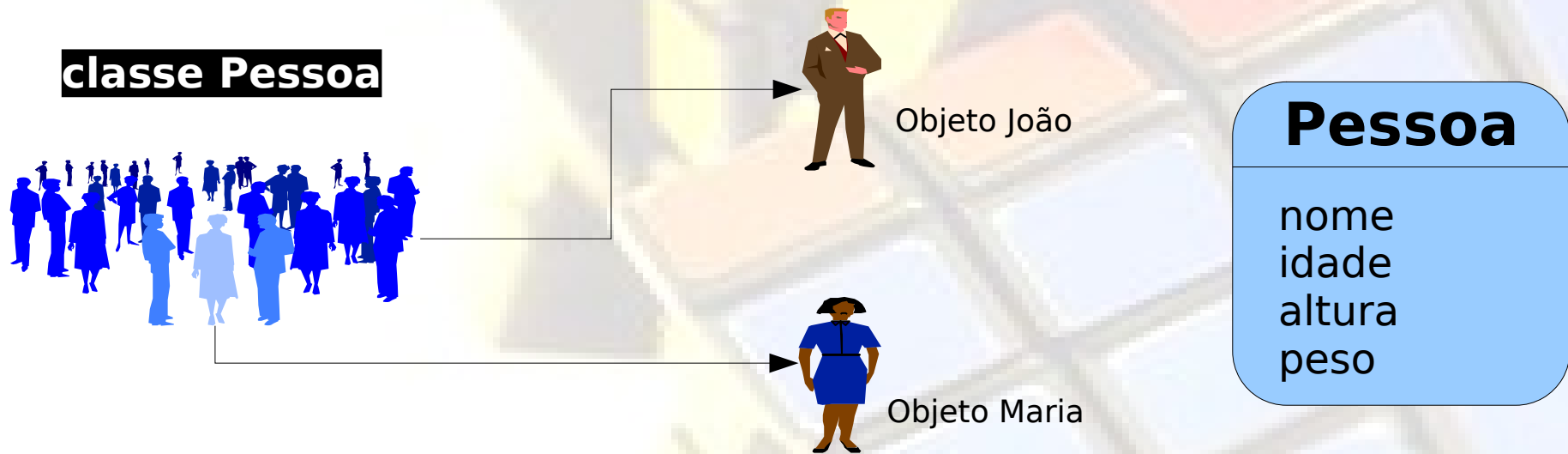


- Uma classe define os atributos e comportamentos comuns compartilhados por um tipo de objeto.
- Conjunto de objetos similares.
 - Mesmo tipo de informação em seu estado;
 - Mesmo comportamento.

Classe



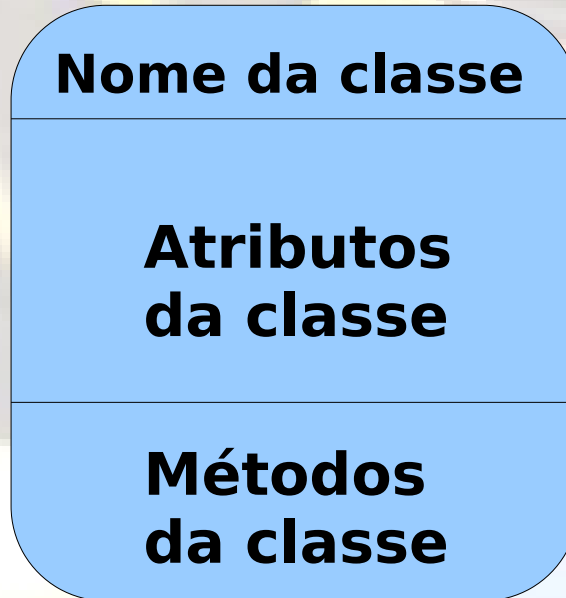
Outros objetos do mesmo tipo também possuem essas características, porém são diferentes uns dos outros.



Classe



Representação gráfica de classe:





- É geralmente descrita como a fôrma a partir da qual um objeto é criado.
- Descreve a:
 - Estrutura de dados;
 - Comportamento de objetos similares.



- Criando uma classe fábrica de **Carros**

- Entidade importante: Carro
- O que **todo** carro tem?
 - Cor
 - Modelo
 - Velocidade atual
 - Velocidade Máxima
- O que **todo** carro faz?
 - Liga
 - Desliga
 - Devolve a marcha engatada
 - Acelera uma quantidade X
 - Breca uma quantidade X

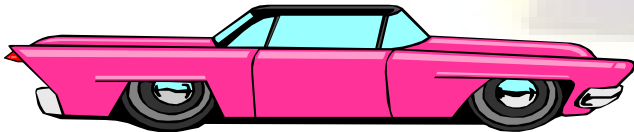


- Criando uma classe fábrica de **Carros**
 - Com isso temos o projeto de um carro.
 - Podemos pegar esse projeto e acelerar sua velocidade?
 - **O que temos ainda é um projeto.** Antes precisamos *construir* um carro para poder acessar o que ele tem. E assim, pedir para ele fazer alguma coisa.
- **CLASSE** (de uma vez por todas!!!): Ao projeto do carro, isto é, à definição do carro damos o nome de **classe**.
 - O que podemos construir a partir desse projeto, que são os carros de verdade, damos o nome de **objeto**.
 - Esse processo de construção de objetos damos o nome de **instanciação** (criação de instâncias) de classes.

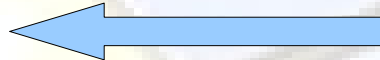
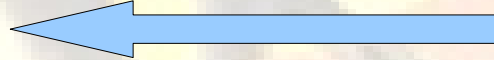
Classe



Objeto 1



Objeto 2



Classe Carro

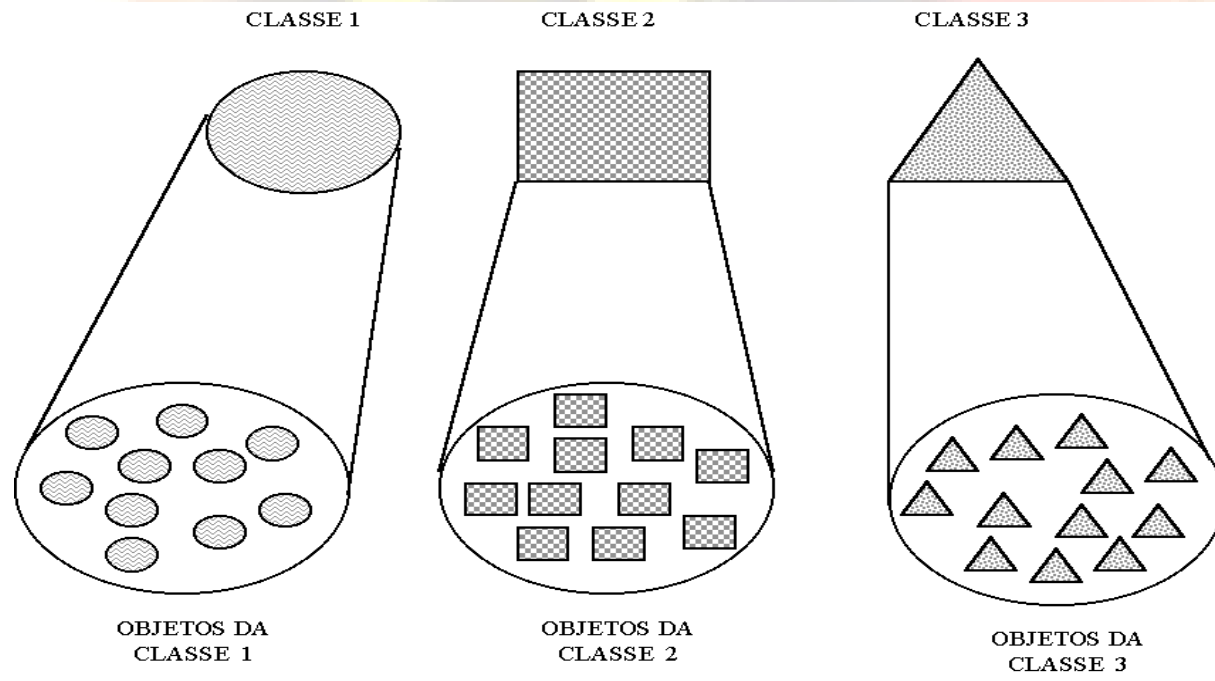
cor
modelo
veloc. Atual
veloc. Máxima

Ligar
Desligar
Acelerar
Brecar

Instância



Cada objeto criado a partir de uma classe é denominado de instância dessa classe.



Classe x Instância

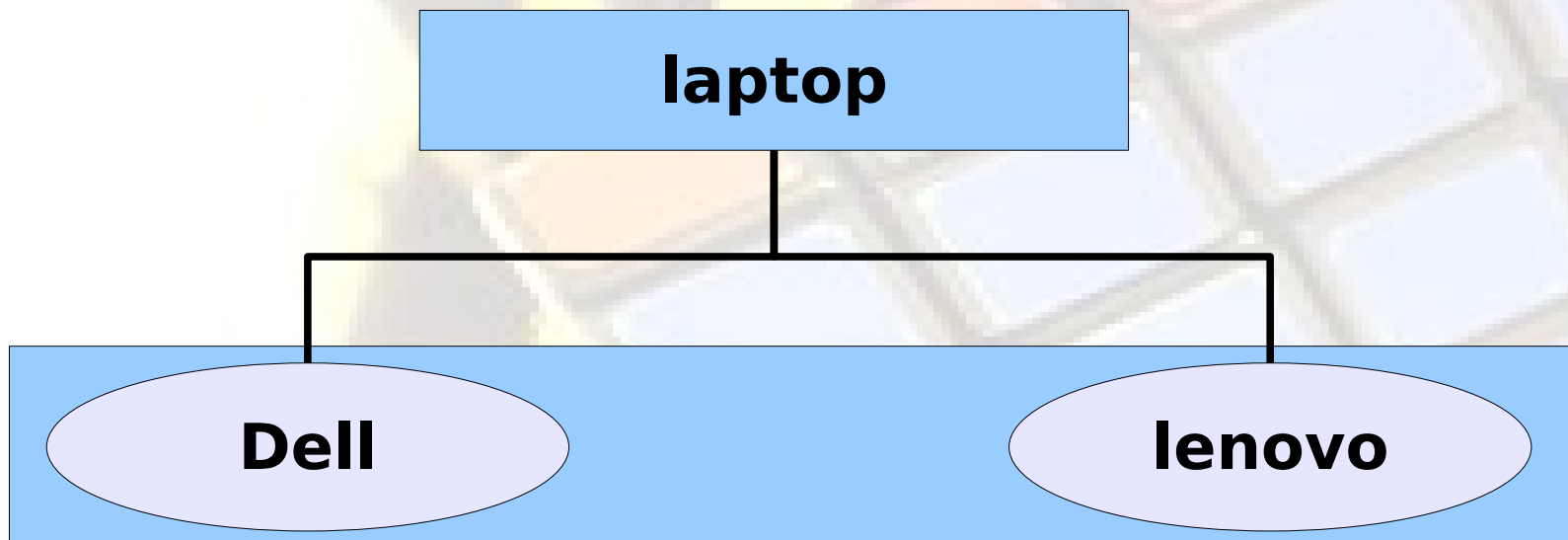


- Cada objeto instanciado a partir de uma classe possui as propriedades e comportamento definidos na classe.
- Classe Funcionario: os funcionários Fernando e Ricardo.
- Classe Carro de Passeio: O Corsa verde placa LCW 1666.
- Classe Nota Fiscal: A nota fiscal #21234 referente a um gaveteiro de escritório e uma cadeira.

Classificação/Instanciação



A capacidade de classificar objetos (em classes) permite expressar relações do tipo classificação/instanciação.



Classificação/Instanciação



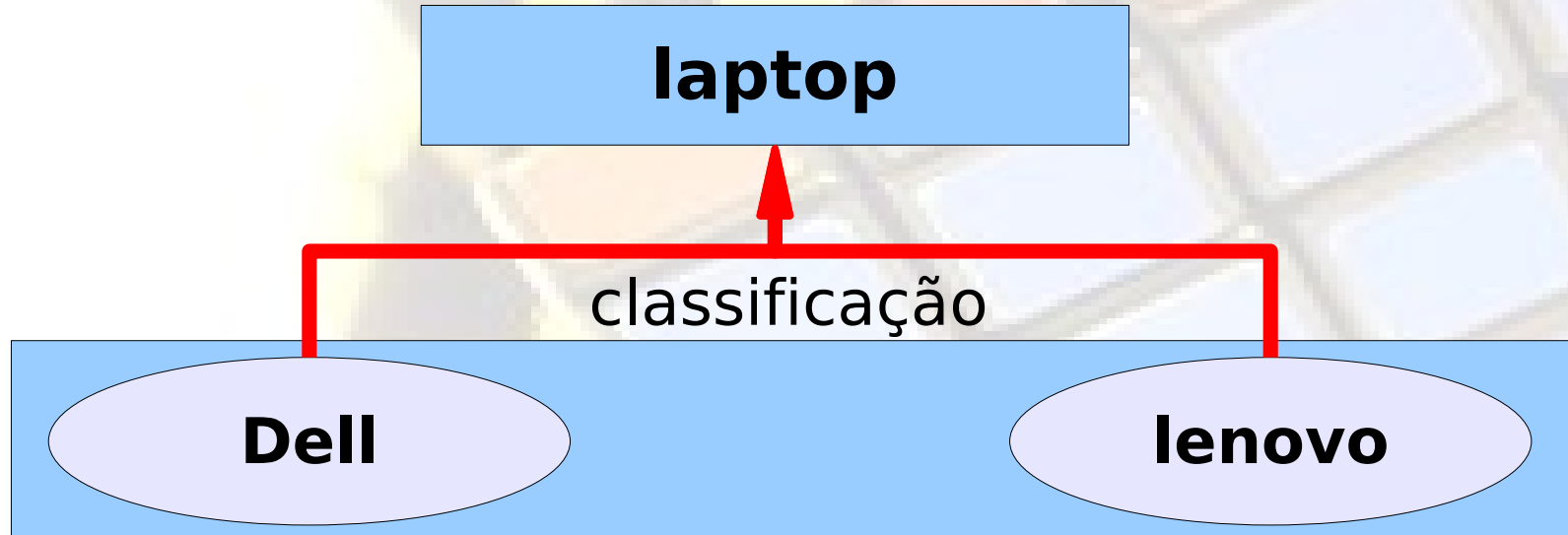
A capacidade de classificar objetos (em classes) permite expressar relações do tipo classificação/instanciação.





Classificação/Instanciação

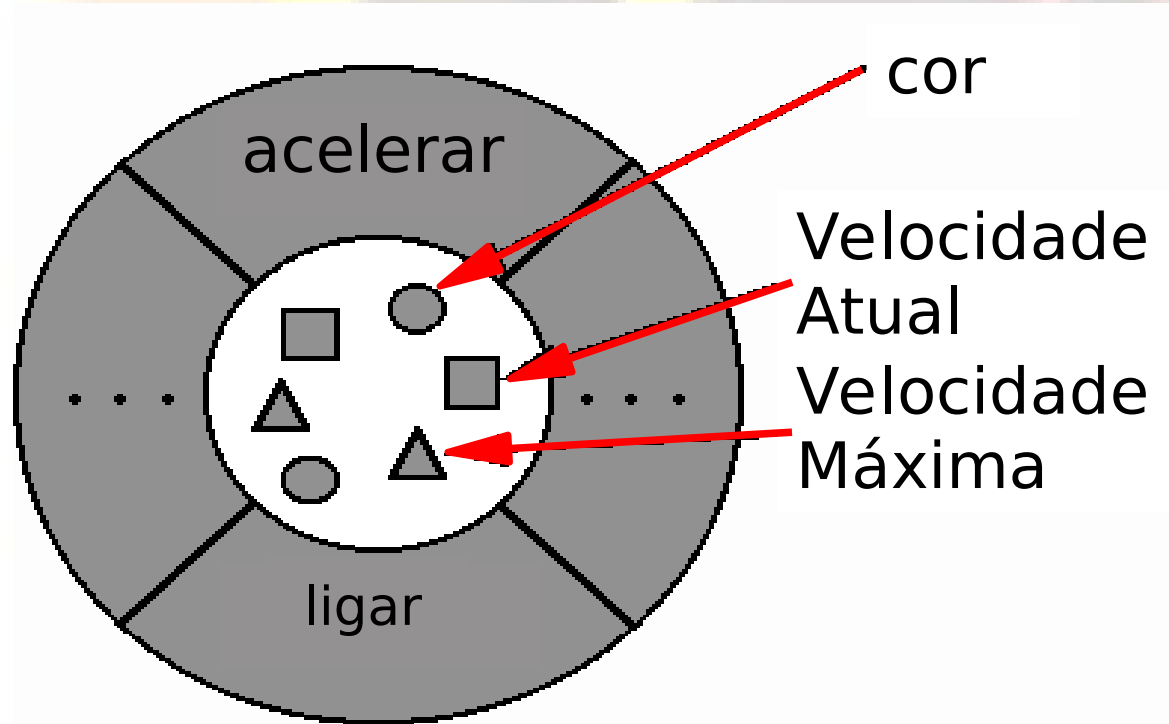
A capacidade de classificar objetos (em classes) permite expressar relações do tipo classificação/instanciação.



Encapsulamento



Ex.: Objeto “meu carro”



Encapsulamento



- Dizemos que um objeto em um programa “encapsula” todo o estado e o comportamento.
 - Tratar o objeto como uma coisa só e ocultar a implementação dos dados do usuário.
 - Objetos encapsulados corretamente são independentes do programa onde são utilizados.
- Proporciona **modularidade** e **ocultamento de informação**.



Objeto = atributos +
métodos +
encapsulamento

Encapsulamento



- Para se garantir o correto encapsulamento de atributos e métodos utiliza-se modificadores de acesso.
- Eles podem ser:
 - Públicos: utilizando-se a palavra **public**
 - Privados ou particulares: palavra **private**

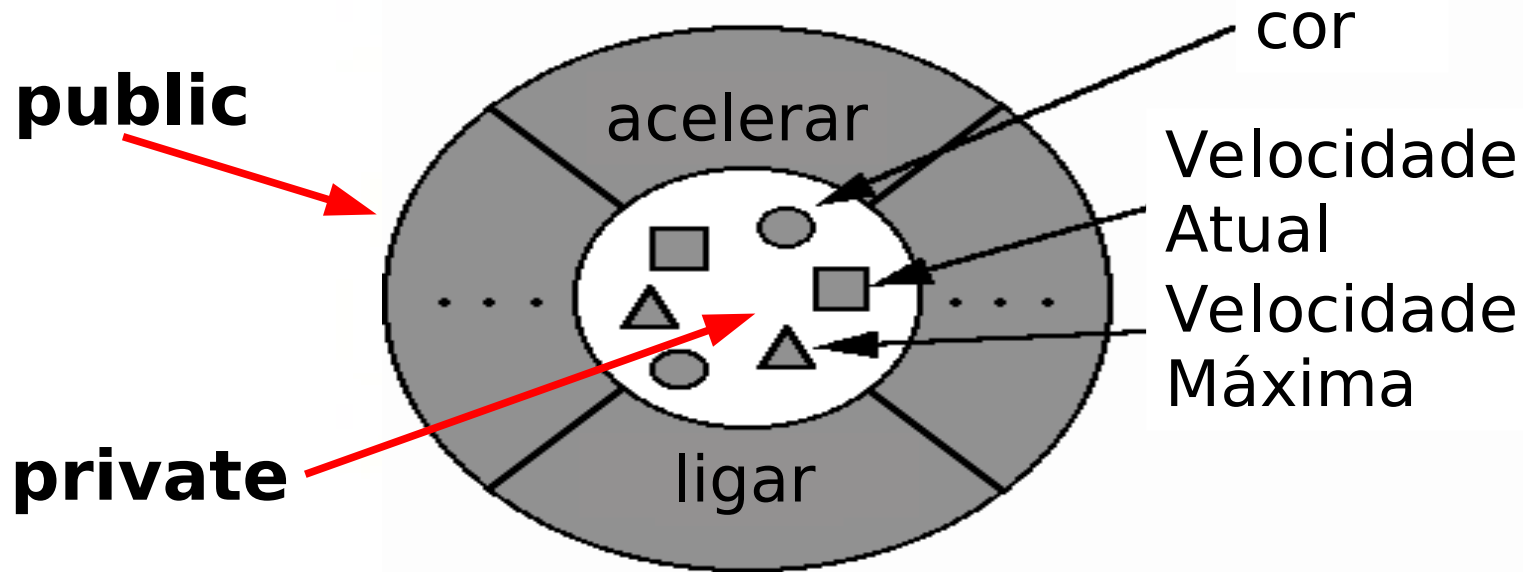
Encapsulamento



- Modificador **private**:
 - Não permite que os atributos ou métodos sejam acessados fora da classe a qual pertencem.
- Modificador **public**:
 - Permite acesso livre ao atributo/método.
- Exemplo: aparelho de DVD
 - Bandeija, conector de saída TV/Som, menu de opções: **public**.
 - Lente de laser, placas de circuito interno: **private**.

Encapsulamento

Ex.: Objeto “meu carro”



Exemplo



A classe Carro em C++: **Carro.h** (Exemplo1)

```
class Carro {  
    private:  
        char cor[20];  
        char modelo[30];  
        float velocAtual;  
        float velocMaxima;  
        bool ligado;  
  
    public:  
        void liga () {  
            velocAtual = 0;  
            ligado = true;  
        }  
  
        void desliga () {  
            velocAtual = 0;  
            ligado = false;  
        }  
};
```



Exemplo

Utilização da classe Carro em C++:

```
#include <iostream>
#include "Carro.h"

int main(void)
{
    Carro meuCarro;

    //PERIGO: acesso a atributo privado
    meuCarro.velocMaxima = 300;
    meuCarro.velocAtual = 50;

    meuCarro.liga();    //invocação do método liga()

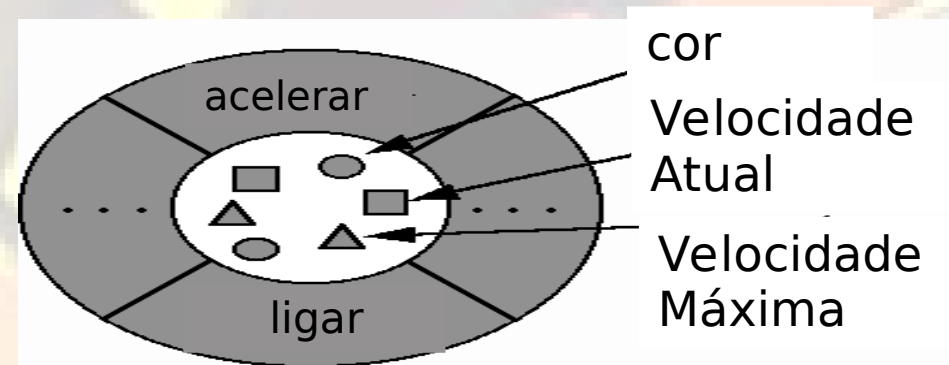
    return 0;
}
```

Em POO os atributos devem ser privados e acessados via métodos públicos!!!

Modificadores de acesso



- Atributos devem ser acessados por métodos públicos!



- O desenvolvedor decide quais atributos podem ser acessados e/ou modificados.
 - Métodos de acesso: prefixo **GET**
 - Métodos de ajuste: prefixo **SET**

Exemplo



A classe Carro em C++: **Carro.h**

```
class Carro {  
    private:  
        char cor[20];  
        char modelo[30];  
        float velocAtual;  
        float velocMaxima;  
        bool ligado;  
    public:  
        void setVelocAtual(float novaVeloc) {  
            velocAtual = novaVeloc;  
        }  
        float getVelocAtual() {  
            return velocAtual;  
        }  
        void setVelocMaxima(float novaVeloc) {  
            velocMaxima = novaVeloc;  
        }  
        float getVelocMaxima() {  
            return velocMaxima;  
        }  
};
```




Exemplo

Utilização da classe Carro em C++:

```
#include <iostream>
#include "Carro.h"
using namespace std;

int main(void) {
    Carro meuCarro;

    //invocação do método público de acesso
    meuCarro.setVelocMaxima(300);
    meuCarro.setVelocAtual(50);

    cout<< "Velocidade máxima: " << meuCarro.getVelocMaxima();

    return 0;
}
```

Mas que tipo de carro criamos???

Sem modelo... sem cor... sem marcado de velocidade...

Construtores



- É o primeiro método invocado no momento da instanciação de um objeto.
- É um método público, sem tipo de retorno e com o mesmo nome da classe.

exemplos

```
class Carro {  
    private: char cor[20];  
            char modelo[30];  
            float velocAtual;  
            float velocMaxima;  
            bool ligado;  
  
    public:  
        Carro ( ) {  
            ligado = false;  
            strcpy(cor, "branco");  
            strcpy(modelo, "FIAT");  
            velocAtual = 0;  
            velocMaxima = 100;  
        }  
  
    ...  
}
```



Construtores

Utilização da construtores:

```
#include <iostream>
#include "Carro.h"
using namespace std;

int main(void) {
    Carro meuCarro; //o construtor é invocado

    cout<< "Velocidade máxima: " << meuCarro.getVelocMaxima();

    return 0;
}

// Saída:
// Velocidade máxima: 100
```

Que tipo de carro criamos???

Sem modelo... sem cor... sem marcado de velocidade...



Sobrecarga de Construtores

É possível sobrecarregar construtores, ou seja, definir outros construtores contendo parâmetros.

```
class Carro {  
    private:  char cor[20];  
              char modelo[30];  
              float velocAtual;  
              float velocMaxima;  
              bool ligado;  
  
    public:  
        Carro ( char novaCor[20], char novoMod[30], float novaVMaxima ) {  
            ligado = false;  
            strcpy(cor, novaCor);  
            strcpy(modelo, novoMod);  
            velocAtual = 0;  
            velocMaxima = novaVMaxima;  
        }  
    . . .  
}
```



Exemplo



Utilização da classe Carro em C++:

```
#include <iostream>
#include "Carro.h"
using namespace std;

int main(void)
{
    Carro c1("vermelho", "GOL", 500);

    cout<< "Velocidade máxima: " << meuCarro.getVelocMaxima();

    return 0;
}

// Saída:
// Velocidade máxima: 500
```



A classe Carro em C++: **Carro.h** (exemplo 4)



```
class Carro {
private: char cor[20];
        char modelo[30];
        float velocAtual;
        float velocMaxima;
        bool ligado;
public: Carro () { ... }
       Carro ( char novaCor[20], char novoMod[30], float novaVMaxima ) {...}

       void liga () { velocAtual = 0; ligado = true; }
       void desliga () { velocAtual = 0; ligado = false; }

       void acelera (float quantidade) {
           if (ligado && velocAtual < velocMaxima)
               if(velocAtual + quantidade < velocMaxima)
                   velocAtual += quantidade;
               else velocAtual = velocMaxima;
       }
       void breca (float quantidade) {
           if (ligado && velocAtual > 0)
               if(velocAtual - quantidade >= 0)
                   velocAtual -= quantidade;
               else velocAtual = 0;
       }
};
```

Exemplo de utilização

mensagens

```
#include <iostream>
#include "Carro.h"
using namespace std;
int main(void)
{
    Carro meuCarro("vermelho", "GOL", 500);
    meuCarro.liga();
    meuCarro.acelera(50); meuCarro.acelera(30); meuCarro.breca(10);

    cout<< "Modelo: " << meuCarro.getModelo() << "\n";
    cout<< "Cor: " << meuCarro.getCor() << "\n";
    cout<< "Velocidade Máxima: " << meuCarro.getVelocMaxima() << "\n";
    cout<< "Velocidade Atual: " << meuCarro.getVelocAtual() << "\n\n";

    cout<< "Pisando fundo..." << "\n";
    meuCarro.acelera(550);
    cout<< "Velocidade Atual: " << meuCarro.getVelocAtual() << "\n\n";

    cout<< "Parando o carro..." << "\n";
    meuCarro.breca(450);
    cout<< "Velocidade Atual: " << meuCarro.getVelocAtual() << "\n\n";

    cout<< "Desligando o carro..." << "\n";
    meuCarro.desliga();
    cout<< "Velocidade Atual: " << meuCarro.getVelocAtual() << "\n";
    return 0;
}
```

Mensagens



- Objetos de software interagem e comunicam-se com outros através de mensagens.
- Quando o objeto A deseja que o objeto B execute um de seus métodos, o objeto A envia uma mensagem ao objeto B.





Mensagens

- Uma mensagem é formada por três componentes básicos:
 - O objeto receptor
 - O nome do método que se deseja executar
 - Os parâmetros (se existirem) necessários ao método.



Relacionamento entre objetos



- Objetos podem ter outros objetos como atributos (estado).
 - Isso indica um relacionamento entre os objetos (todo-parte), que pode ser:
 - Dependência;
 - Agregação;
 - Composição.

Relacionamento entre objetos



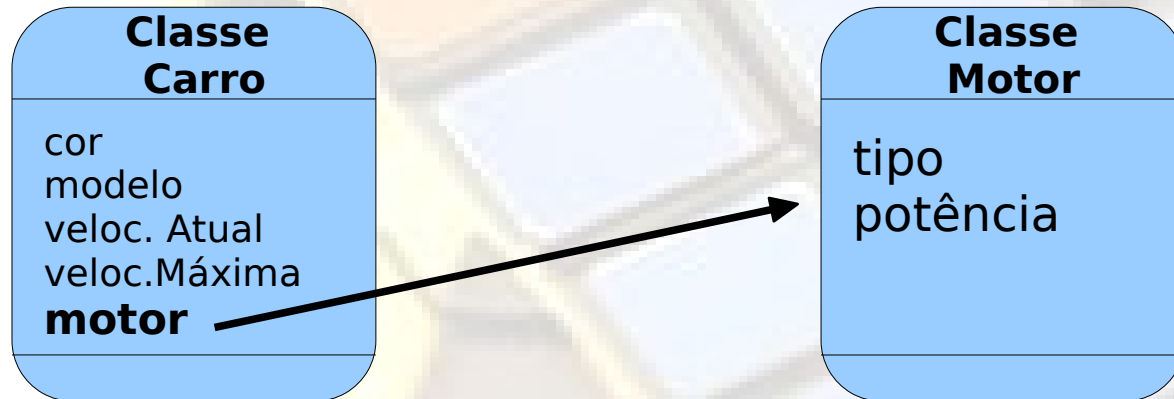
- Relacionamento de dependência:
 - Tipo mais fraco;
 - Indica uma relação temporária entre os objetos.
 - Ex.: acesso a um caixa-eletrônico bancário.
- Em c++:

```
void metodo1 ( Classe2 obj2 ) {  
    obj2.metodo2();  
}
```

Relacionamento entre objetos



- Relacionamento de agregação:
 - Tipo mais forte que dependência;
 - Indica uma relação duradoura entre os objetos (todo-parte).
 - As partes são independentes do todo.
 - Ex.: um carro utiliza um motor para funcionar.





Exemplo em C++: classe Motor (exemplo 5)

```
class Motor
{
    private:
        int potencia;
        char tipo[10];
    public:
        Motor ( ) {
            potencia = 0;
            strcpy( tipo, "0v" );
        }
        Motor ( int novaPotencia, char novoTipo[10] ) {
            potencia = novaPotencia;
            strcpy( tipo, novoTipo );
        }

        int getPotencia() { return potencia; }
        char* getTipo() { return tipo; }
};
```

Exemplo



Exemplo em C++: classe Carro

```
class Carro
{
    private:
        char cor[20];
        char modelo[30];
        float velocAtual;
        float velocMaxima;
        bool ligado;
        Motor motor; //atributo motor (agregação)
    public:
        Carro ( char novaCor[20], char novoMod[30], float novaVMax, Motor novoMotor ) {
            ligado = false;
            strcpy(cor, novaCor);
            strcpy(modelo, novoMod);
            velocAtual = 0;
            velocMaxima = novaVMax;
            motor = novoMotor; //novo motor é atribuido
        }

        Motor getMotor() {
            return motor;
        }
};
```

Exemplo



Exemplo em C++: classe UsoCarro (Apêndice 1.4)

```
#include <iostream>
#include "Carro.h"
using namespace std;

int main(void)
{
    Motor meuMotor(500, "16v");

    Carro meuCarro("vermelho", "GOL", 500, meuMotor);
    meuCarro.liga();
    meuCarro.acelera(50);
    meuCarro.acelera(30);
    meuCarro.breca(10);

    cout<< "Tipo do Motor: " << meuCarro.getMotor().getTipo() << "\n";
    cout<< "Potencia do Motor: " << meuCarro.getMotor().getPotencia() << "\n";

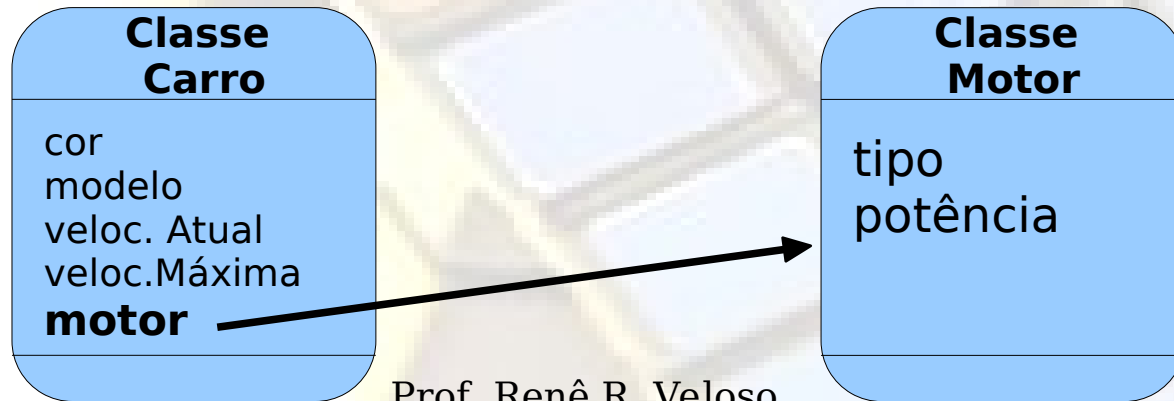
    cout<< "Modelo: " << meuCarro.getModelo() << "\n";
    cout<< "Cor: " << meuCarro.getCor() << "\n";
    cout<< "Velocidade Máxima: " << meuCarro.getVelocMaxima() << "\n";
    cout<< "Velocidade Atual: " << meuCarro.getVelocAtual() << "\n\n";

    . . .
```

Relacionamento entre objetos



- Relacionamento de composição:
 - Tipo mais forte que agregação;
 - Indica uma relação duradoura entre os objetos (todo-parte).
 - As partes são dependentes do todo.
 - Ex.: um carro utiliza um motor para funcionar, mas o motor não existe fora do carro.



Exemplo



Exemplo em C++: classe Carro

(Exemplo 6)

```
class Carro
{
    private:
        char cor[20];
        char modelo[30];
        float velocAtual;
        float velocMaxima;
        bool ligado;
        Motor motor; //atributo motor (composição)
    public:
        Carro ( char nCor[20], char nMod[30], float nVMax, int nPot, char nTipo[10] ) {
            ligado = false;
            strcpy(cor, novaCor);
            strcpy(modelo, novoMod);
            velocAtual = 0;
            velocMaxima = novaVMax;
            motor = Motor(nPot, nTipo); //criação de nova instância
        }

        Motor getMotor() {
            return motor;
        }
};
```



Exemplo

Exemplo em C++: classe UsoCarro

```
#include <iostream.h>
#include "Carro.h"
using namespace std;

int main(void)
{
    //Motor meuMotor();

    Carro meuCarro("vermelho", "GOL", 500, 500, "16v");
    meuCarro.liga();
    meuCarro.acelera(50);
    meuCarro.acelera(30);
    meuCarro.breca(10);

    cout<< "Tipo do Motor: " << meuCarro.getMotor().getTipo() << "\n";
    cout<< "Potencia do Motor: " << meuCarro.getMotor().getPotencia()<< "\n";

    cout<< "Modelo: " << meuCarro.getModelo() << "\n";
    cout<< "Cor: " << meuCarro.getCor() << "\n";
    cout<< "Velocidade Máxima: " << meuCarro.getVelocMaxima() << "\n";
    cout<< "Velocidade Atual: " << meuCarro.getVelocAtual() << "\n\n";

    . . .
```

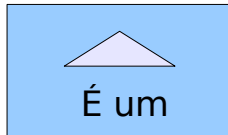
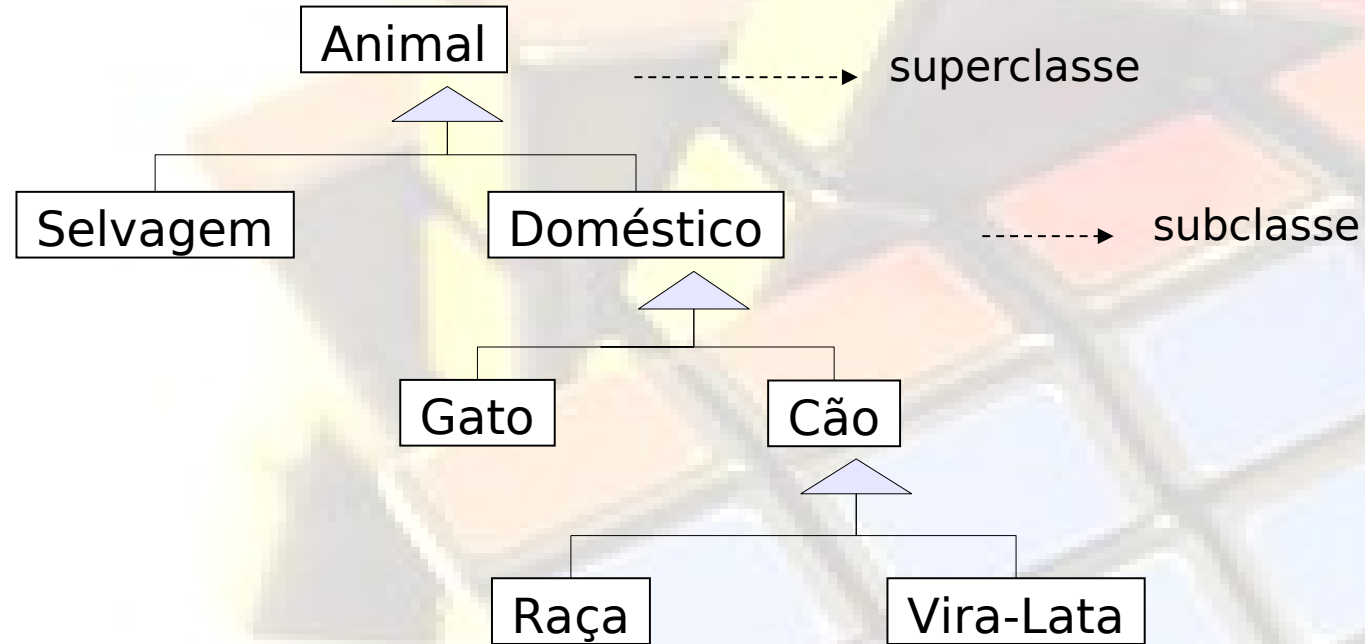


- Uma classe pode ser derivada de outra classe, e desta forma herdar tanto seu estado quanto seu comportamento.
- HERANÇA:
 - Mecanismo que permite definir uma nova classe (subclasse) a partir de uma classe já existente (superclasse).
 - Habilidade de um objeto derivar seus atributos e métodos automaticamente de outro objeto.
- **A nova classe mantém o estado e a funcionalidade da classe da qual deriva; por isso, dizemos que ela “herda” as características daquela classe.**



- Classes podem ser construídas a partir de outras já existentes; facilita a extensibilidade.
- A herança pode ser de dois tipos:
 - **Simples**: quando uma classe é subclasse de somente uma superclasse.
 - **Múltipla**: quando uma classe é subclasse de várias superclasses e conseqüentemente herda as características de cada uma delas.

Herança simples



Objeto Rex

instância da classe Vira-Lata



Rex

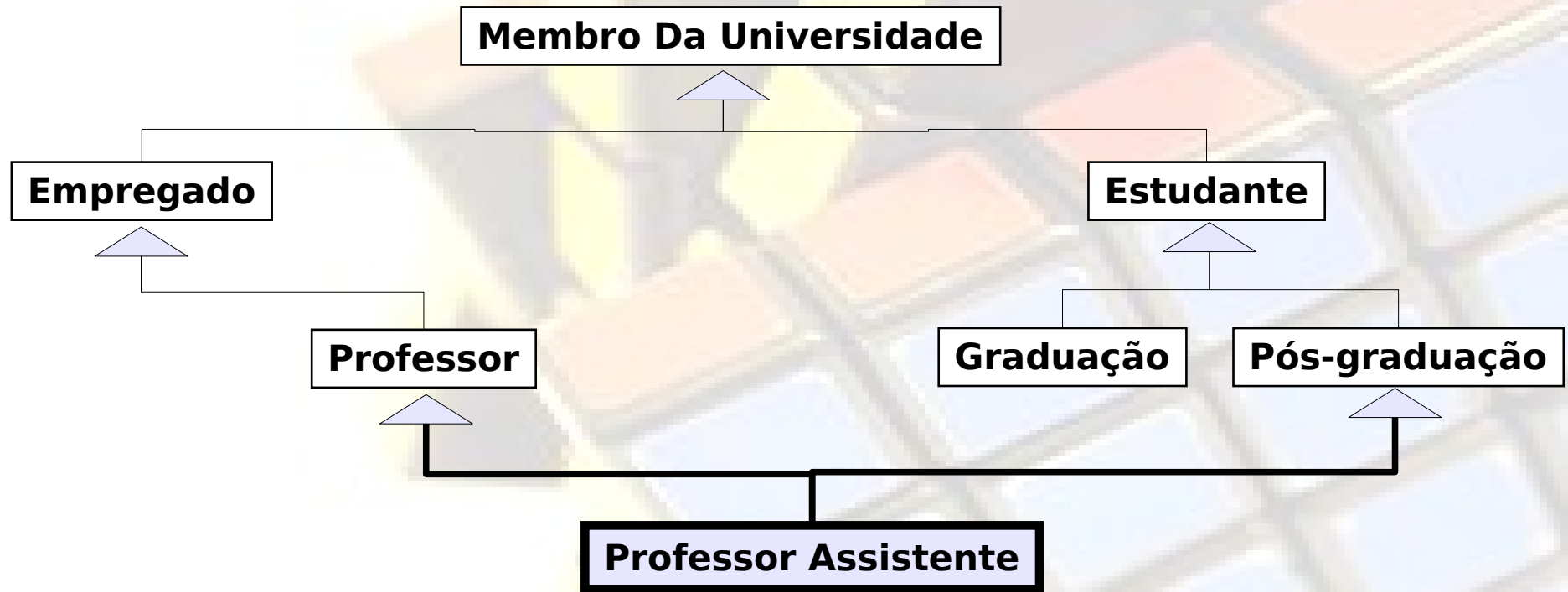


Herança simples

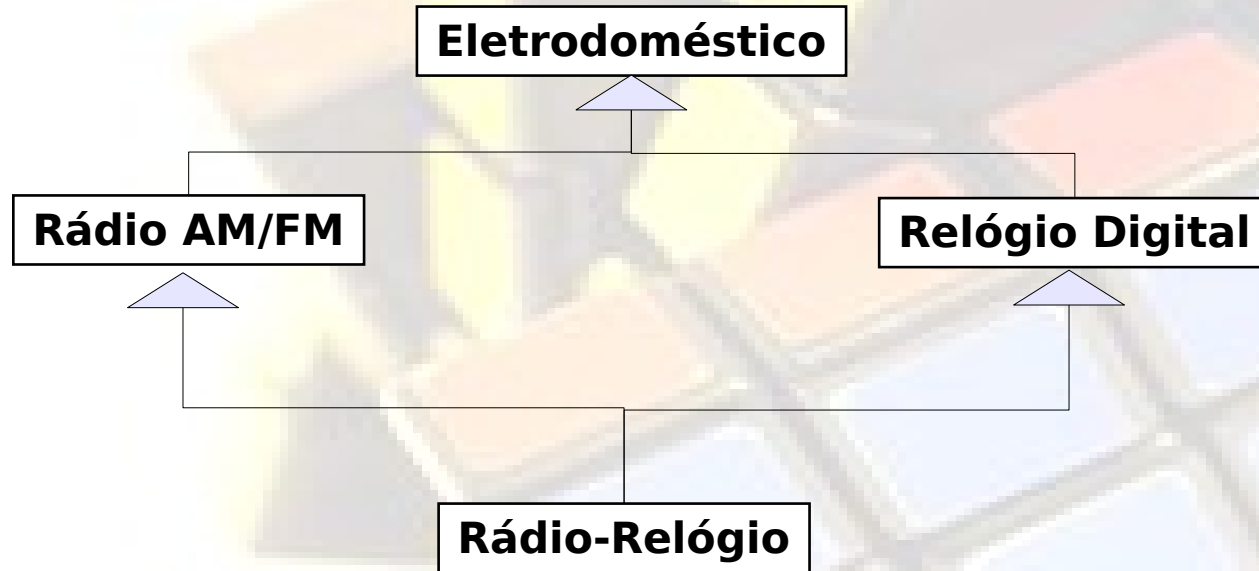
Os relacionamentos de herança formam estruturas hierárquicas do tipo árvore.



Herança Múltipla



Herança Múltipla



Herança

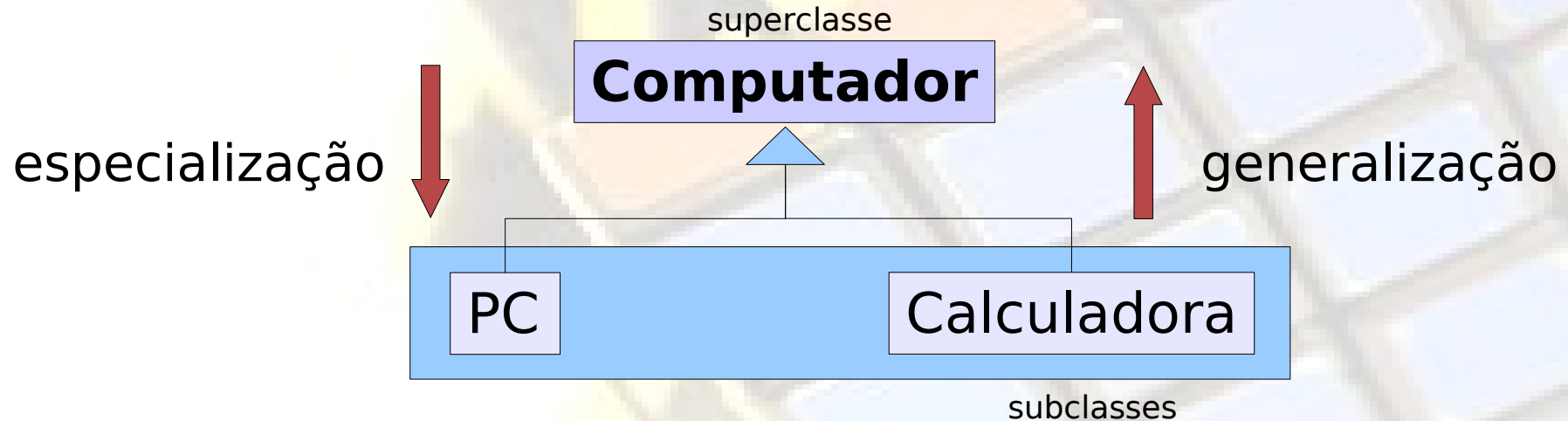


Superclasse	Subclasse
Aluno	AlunoDeGraduação AlunoDePósGraduação
Forma	Círculo Triângulo Retângulo
Financiamento	FinanciamentoDoCarro FinanciamentoDaReformaDaCasa FinanciamentoDaCasa
Empregado	FuncionárioOperacional CorpoDocente
Conta	ContaCorrente ContaPoupança

Generalização X Especialização



O mecanismo de herança pode ser chamado de **especialização**.
A **generalização** ocorre quando, a partir da observação de duas classes, abstraímos dela uma classe mais genérica.

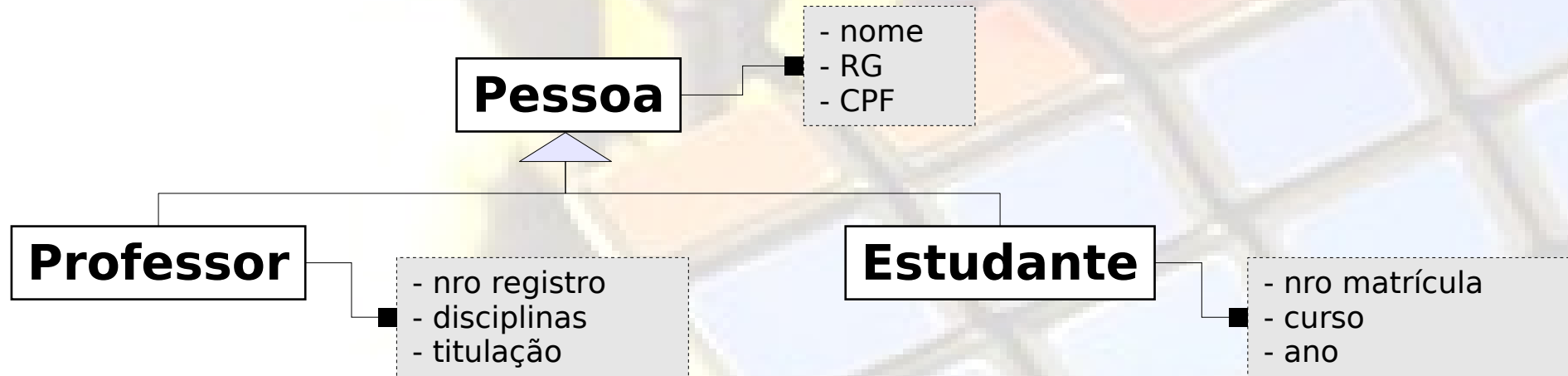




Compartilhamento de características



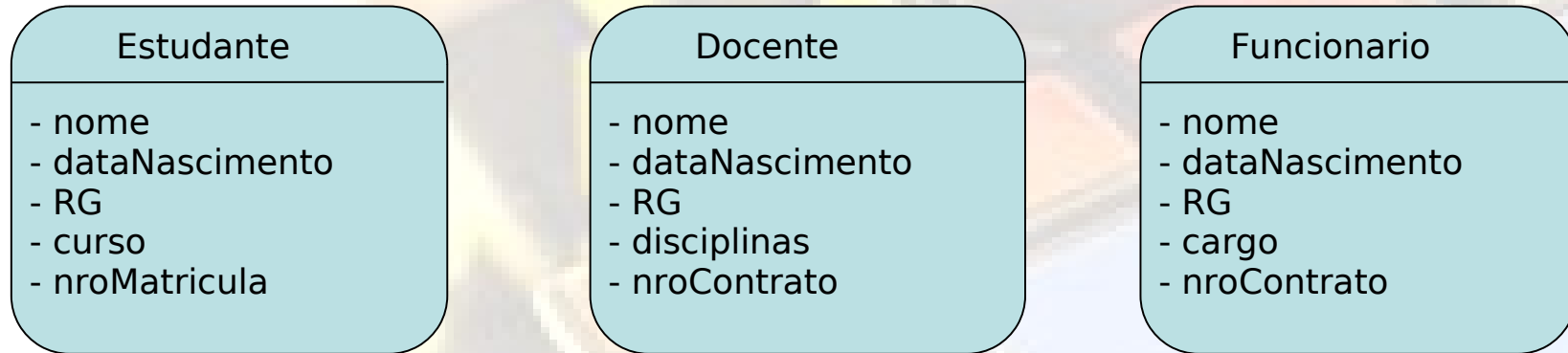
O mecanismo de herança permite que os objetos envolvidos compartilhem seu estado e comportamento com as suas subclasses.



Exemplo I



Sistema de administração universitária.



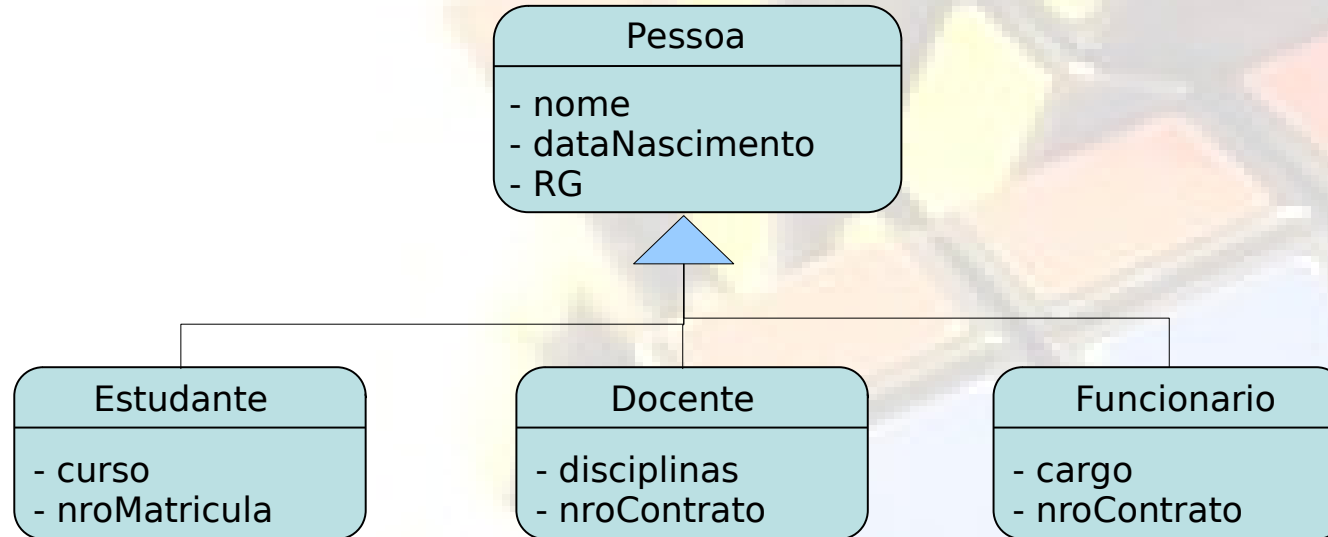
Para adicionar uma nova classe (por exemplo, Vestibulando) teríamos que repetir todos os atributos como: **nome**, **dataNascimento**, **RG** e mais o **nroInscrição** e o **curso**.

Exemplo I



Sistema de administração universitária.

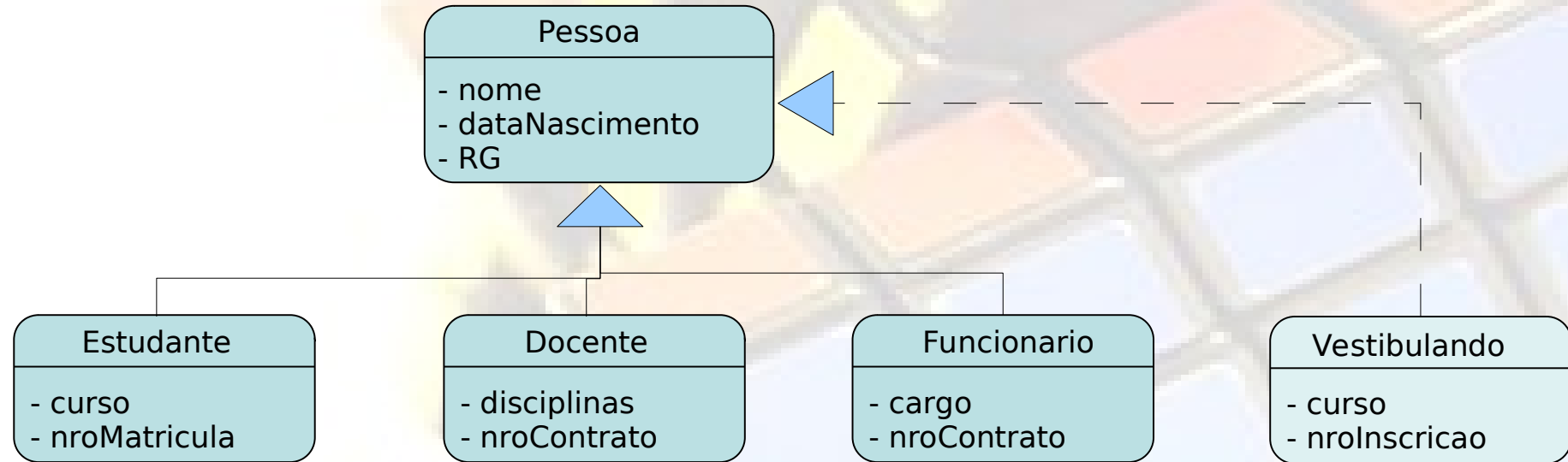
Apêndice 2





Exemplo I

Sistema de administração universitária.



Exemplo II



PetShop – Venda de pássaros

(exemplo 7)

Controle de alimentação, peso e identificação dos pássaros.

```
class Passaro {  
    private:  
        int id;  
        char nome[20];  
        float peso;  
        float preco;  
    public:  
        Passaro(int nId, char nNome[20], float nPeso, float nPreco) {  
            id = nId;  
            strcpy(nome, nNome);  
            peso = nPeso;  
            preco = nPreco;  
        }  
  
        int getId() { return id; }  
        char* getNome() { return nome; }  
        float getPeso() { return peso; }  
        float getPreco() { return preco; }  
};
```

Passaro

- id
- nome
- peso
- preco

Exemplo II



Além de uma ave comum, vendemos também aves de outros países. É necessário saber sua origem e um código que o IBAMA usa para rastrear os animais que chegam do exterior.

```
class PassaroImportado {
private: int id;
        char nome[20];
        float peso, preco;
        char paisDeOrigem[30];
        char codigoIbama[10];

public:
    PassaroImportado(int nId, char nNome[20],
                     float nPeso, float nPreco, char pais[30]) {
        id = nId;
        strcpy(nome, nNome);
        peso = nPeso; preco = nPreco;
        strcpy(paisDeOrigem, pais);
    }
    void legaliza(String nCodigoIbama) {
        strcpy(codigoIbama, nCodigoIbama);
        cout<<"Passaro "<< nome <<" foi legalizado: "<< codigoIbama;
    }

    //outros métodos de acesso parecidos com os da classe Passaro
};
```

PassaroImportado

- id
- nome
- peso
- preco
- paisDeOrigem
- codigoIbama

Exemplo II



- ♦ Problemas...
 - ♦ Se tivéssemos um outro tipo de pássaro, que possui algumas características diferentes da ave comum, precisaríamos criar uma outra classe e copiar o código novamente!
 - ♦ Além disso, se um dia necessitarmos adicionar uma nova informação a todos os pássaros, deveríamos passar por todas as classes de pássaros e adicionar este atributo.
 - ♦ **O problema ocorre por não centralizar as informações principais do pássaro em um único lugar.**



Exemplo II

Solução...

Gostaríamos de fazer com que o `PassaroImportado` tivesse tudo que um `Passaro` tem, ou seja, que fosse uma extensão de `Passaro`.

Exemplo (operador `:`)...

```
class PassaroImportado : public Passaro
{
    .
    .
    .
}
```

Herança em subclasses



```
class PassaroImportado : public Passaro{
private:
    char paisDeOrigem[30];
    char codigoIbama[10];
public:
    PassaroImportado(int nId, char nNome[20], float nPeso, float nPreco,
                     char pais[30])
        : Passaro(nId, nNome, nPeso, nPreco)
    {
        strcpy(paisDeOrigem, pais);
    }

    void legaliza(char nCodigoIbama[10]) {
        strcpy(codigoIbama, nCodigoIbama);
        cout<<"Passaro "<< nome <<" foi legalizado com codigo "<< codigoIbama;
    }
}
```

Um objeto do tipo **PassaroImportado** possuirá também os atributos definidos na classe **Passaro**, pois agora um **PassaroImportado** é um **Passaro**.

Obs.: O construtor de **PassaroImportado** pode invocar o construtor de **Passaro**.



Construtores em subclasses

Quando um objeto de uma subclasse é instanciado, o construtor da superclasse deve ser chamado para fazer qualquer inicialização necessária dos seus atributos.

Uma chamada explícita ao construtor da superclasse pode ser fornecida dentro do construtor da subclasse via operador ':':

```
class SuperClasse {  
    SuperClasse() {  
        ...  
    }  
}  
  
class SubClasse : public SuperClasse  
{  
    SubClasse() : SuperClasse()  
    {  
        ...  
    }  
}
```

```
class SuperClasse {  
    SuperClasse(int par) {  
        ...  
    }  
}  
  
class SubClasse : public SuperClasse  
{  
    SubClasse(int par1, int par2) : SuperClasse(par1)  
    {  
        ...  
    }  
}
```

Visibilidade *protected*



- Dizemos que a classe **PassaroImportado** herda todos os atributos e métodos da classe mãe, no nosso caso, o **Passaro**.
 - Usamos o tipo de visibilidade ***private***.
 - As subclasses não herdam atributos e métodos ***private***.
 - Não podemos deixar os atributos como ***public***.
- Existe um outro modificador de acesso, o ***protected***, que fica entre o ***private*** e o ***public***.
- Um atributo ***protected*** só pode ser acessado (visível) pela própria classe ou suas subclasses.



Visibilidade *protected*

```
class Passaro
{
  protected:
  int id;
  char nome[20];
  float peso;
  float preco;

  //métodos
}
```

Sempre usar `protected` ???

É uma boa idéia deixar que a classe filha acesse todos os atributos da classe mãe?

Reescrita de método (overriding)



No nosso modelo, o preço guardado na classe **Passaro** é o valor de custo. Nossa empresa tem a seguinte regra:

O valor de venda de um pássaro importado é 20% a mais que o custo devido aos impostos cobrados pela importação.

```
class Passaro {  
    protected:  
        float preco;  
        //...  
    public:  
        //...  
        float getPreco() {  
            return preco;  
        }  
        //...
```

Se deixarmos a classe **PassaroImportado** como está, ela vai herdar o método **getPreco()**. Retornando o preço de custo sem os devidos impostos.



Reescrita de método (overriding)

```
PassaroImportado passaroI(1010, "Aguia", 2, 2000, "EUA");  
  
cout<< "Preco: " << passaroI.getPreco() << "\n";
```

O resultado aqui será **2000**. Não queremos esta resposta, não queremos este método da classe mãe. Queremos reescrever (**sobrepôr, override**) este método.

```
class PassaroImportado : public Passaro  
{  
    //...  
    public:  
    //...  
  
    float getPreco() { return (preco * 1.20); }  
  
    char* getPaisDeOrigem() { return paisDeOrigem; }  
    char* getCodigolbama() { return codigolbama; }  
};
```




Sobrecarga de método (overloading)



- Em OO, métodos podem ter o mesmo nome desde que não sejam ambíguos.
 - Diferem-se pelo tipo e/ou quantidade de parâmetros.
- Isso se chama **sobrecarga** de método (**overloading**). Não confundir com **overriding**, que é um conceito muito mais importante.

```
...  
float getPreco() {  
    return (preco * 1.20);  
}  
  
float getPreco(float perc) {  
    return (preco * perc);  
}  
...
```

Obs.:

Os métodos sobrecarregados devem possuir mesmo tipo de retorno.

Portanto, aplica-se somente aos seus parâmetros.

Nossa PetShop resolveu trabalhar com gatos também...

```
class Gato
{
  protected:
    int id;
    char nome[20];
    float peso;
    float preco;

    //métodos
};
```

O que aconteceria no dia em que o cliente pedisse a informação de quanto tempo em que o animal se encontra na loja?

Teríamos que adicionar essa informação tanto em **Gato** quanto em **Passaro**...

Uma vez que **Gato** é muito parecido com **Passaro**, poderíamos então usar herança, como por exemplo fazer o **Gato** herdar de **Passaro**???

Solução...

Uma boa solução é criar uma classe **Animal** onde concentramos as informações básicas de um animal qualquer...

Agora podemos criar duas classes **Gato** e **Passaro** herdando a classe **Animal** e todas as suas características.

```
class Animal
{
    protected:
        int id;
        char nome[20];
        float peso;
        float preco;

        //métodos
};
```

```
class Gato : public Animal {
    protected:
        char tipoRacao[20];
        //...
};

class Passaro : public Animal {
    protected:
        bool canta;
        //...
};
```



Herança - Polimorfismo

- O que exatamente vem a ser a nossa classe Animal?
 - Vendemos apenas pássaros e gatos...
 - Faz algum sentido termos uma classe Animal?
- Estamos usando esta classe apenas para fornecer os atributos em comum dos animais e para tirar proveito do polimorfismo.
- POLIMORFISMO: técnica de programação OO na qual uma superclasse pode assumir a forma de qualquer subclasse.

Polimorfismo - Exemplo

Exemplo 10

Vamos então herdar de Animal, sobrepondo o método getPreco(), já que a 'precificação' de um gato e de um pássaro são diferentes ao do método herdado:

Um pássaro nacional custa 5% a mais ao preço de custo.

Um pássaro importado custa 20% a mais que um nacional.

Um gato custa 30% a mais que o preço de custo.

```
class Passaro : public Animal {  
    float getPreco() { return (preco * 1.05); }  
};  
  
class PassaroImportado : public Passaro {  
    float getPreco() { return (preco * 1.20); }  
};  
  
class Gato : public Animal {  
    float getPreco() { return (preco * 1.30); }  
};
```

Polimorfismo - Exemplo



Com cada tipo de animal possuindo preço diferente do outro, podemos definir um método no programa principal responsável por processar as vendas da loja.

```
void vendeAnimal(Animal *a)
{
    cout<< "Processando venda de animal...\n";
    cout<< "Nome:" << a->getNome() << "\n";
    cout<< "Preco:" << a->getPreco() << "\n";
    cout<< "...Venda registrada no banco de dados!\n\n";
}
```

```
Gato gato1(1111, "Gato Preto", 5, 200, "carne de sol");
Passaro passaro1(2222, "Canário", 0.100, 50, true);

vendeAnimal(&passaro1);
vendeAnimal(&gato1);
```



Polimorfismo – Exemplo

- ♦ Problemas...
 - ♦ O preço impresso na tela após a execução do exemplo anterior não corresponde ao método sobreposto pelas subclasses.
 - ♦ O polimorfismo não funcionou!!!
- ♦ Solução...
 - ♦ Devemos informar à superclasse que seus métodos serão sobrepostos.
 - ♦ Uso de métodos **VIRTUAIS**.



- Um método que pode ser redefinido em subclasses é conhecido como método virtual.
- De maneira a declarar um método como virtual, devemos preceder sua declaração com a palavra-chave **virtual**.

```
...  
virtual float getPreco() { return preco; }  
...
```

Alterar o exemplo anterior!!!



Polimorfismo

Um erro muito comum é tentar acessar métodos que não estão definidos na referência polimórfica:

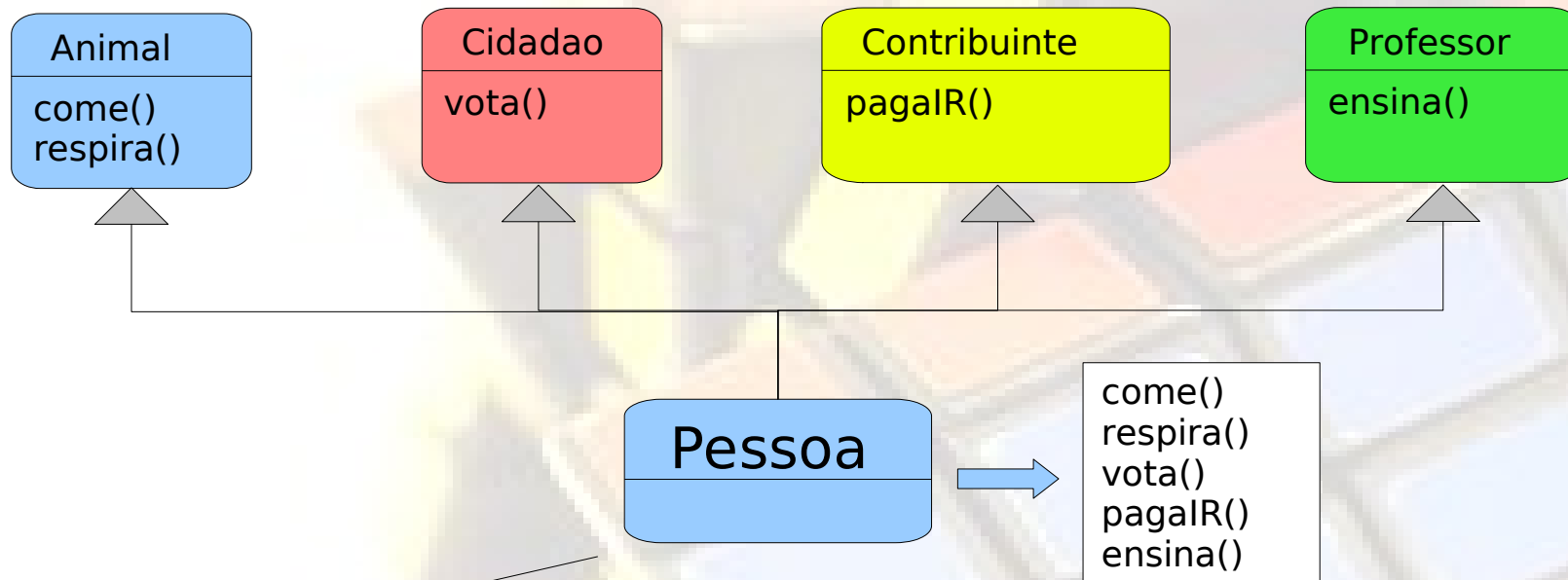
```
void vendeAnimal(Animal *a)
{
    cout<< "Processando venda de animal...\n";
    cout<< "Nome:" << a->getNome() << "\n";
    cout<< "Preco:" << a->getPreco() << "\n";

    cout<< "Tipo de Ração:" << a->getTipoRacao() << "\n";

    cout<< "...Venda registrada no banco de dados!\n\n";
}
```

ERRO!!! Não é um método de Animal.

Polimorfismo em herança múltipla



Este objeto pode ser usado em qualquer lugar onde for aceito um Animal, um Cidadão, um Contribuinte e/ou um Professor.

Exemplo



```
class Pessoa : public Animal, public Cidadao, public Contribuinte,
               public Professor
{
    public:
        void come() { /* come */ }
        void respira() { /* respira */ }
        void vota() { /* vota */ }
        void pagaIR() { /* paga IR */ }
        void ensina() { /* ensina */ }
};
```

```
class Cidade
{
    public:
        void alimenta(Animal A) { /* alimenta A */ }
        void convoca(Cidadao Ci) { /* convoca Ci */ }
        void cobraDe(Contribuinte Co) { /* recebe de Co */ }
        void contrata(Professor P) { /* paga P */ }
};
```



Exemplo

```
int main(void)
{
    Pessoa joao;
    Cidade moc;

    moc.contrata(joao); //considera Professor
    moc.cobraDe(joao); //considera Contribuinte
    moc.convoca(joao); //considera Cidadao
    moc.alimenta(joao); //considera Animal
}
```

O objeto **joao** pode assumir diferentes formas dependendo do método do objeto **moc** invocado.

Conclusão



- ♦ A programação Orientada a Objetos ainda é a melhor maneira de se construir software.
- ♦ Melhora o entendimento do problema.
 - ♦ Alto nível de abstração.
 - ♦ Possui conceitos claros e bem definidos.
 - ♦ Facilita no processo de documentação.
- ♦ Divide o problema facilitando o compartilhamento de responsabilidades aos desenvolvedores.
- ♦ Facilita o reuso de partes do software.

Referências



<http://www.mundooo.com.br/>

<http://www.oopweb.com/>

<http://www.cplusplus.com/>

<http://www.espacoacademico.com.br/035/35amsf.htm>

<http://www.toa.com/pub/oobasics/oobasics.htm>

<http://www.zib.de/visual/people/mueller/Course/Tutorial/tutorial.html>

<http://csis.pace.edu/~bergin/patterns/ppoop.html>

<http://www.student.chula.ac.th/~46829016/history.htm>